Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Eşzamanlılık Desenleri ve OBYS

Bölüm 11 – Hafta 11

Yazılım Geliştirme II (BILB2000)

Bahar Dönemi – 11. Hafta

Bu Haftanın Planı

1. Ünite 1: Thread-Safe Veri Yapıları
2. Ünite 2: Eşzamanlı Dosya İşlemleri
3. Ünite 3: İlerleme Takibi ve İptal
4. Ünite 4: OBYS Mini Proje

Hafta 11'in Büyük Resmi

Eşzamanlılık becerilerimizi OBYS'ye uyguluyoruz

Bölüm 9-10'da `thread`, `mutex`, `condition_variable`, `async` ve QThread'i öğrendik. Bu hafta bu primitifleri **tasarım kalıplarına** dönüştürüyoruz: thread-safe veri yapıları, paralel I/O, ilerleme/iptal, ve OBYS'nin responsive sürümü.

Kazanımlar

- ThreadSafeQueue<T> tasarımı
- Sharded ConcurrentMap
- shared_mutex okuyucu/yazıcı kalıbı
- Paralel CSV/JSON I/O
- C++20 stop_token, jthread
- QProgressDialog entegrasyonu

OBYS Bu Hafta

- Tüm I/O arka planda
- UI hiç donmaz
- Eş zamanlı çoklu işlem
- İptal mekanizması
- Thread-safe Depo
- Profesyonel kategori

Ünite 1: Thread-Safe Veri Yapıları

Ünite 1 – Öğrenme Hedefleri

Bu ünitede öğreneceğin

- Arayüz yarışı (interface race) kavramı
- ThreadSafeQueue<T> tasarımı
- Sharded ConcurrentMap
- shared_mutex ile okuyucu/yazıcı kilidi
- OBYS `Depo<T>`'nun thread-safe sürümü

Arayüz Yarışı: Klasik Tuzak

```
std::queue<int> kuyruk;
std::mutex m;

if (!kuyruk.empty()) {          // Thread A: false degil
    // Thread B aynen pop yapti!
    int x = kuyruk.front();     // Bos kuyruktan okuma -> UB
    kuyruk.pop();
}
```

Sorun

Bireysel metodlar kilitli olsa da **ardışık çağrı kombinasyonu** yarış üretir.

Çözüm

Yarış üreten metod kombinasyonlarını **tek atomik metoda** indirge: `tryPop`.

ThreadSafeQueue: tryPop ve waitAndPop

```
template <typename T>
class ThreadSafeQueue {
    mutable std::mutex m_;
    std::condition_variable cv_;
    std::queue<T> kuyruk_;
    bool kapali_ = false;
public:
    void push(T d);
    std::optional<T> tryPop();          // Bossa nullopt
    bool waitAndPop(T& cikti);          // Bekler; kapanista false
    void kapat();                       // notify_all
};
```

- Kopyalama yasak (= `delete`)
- `kapat()` bayrak + `notify_all`
- `waitAndPop` predicate ile sahte uyanmaya karşı korumalı

Sharded ConcurrentMap: Tek Mutex Darboğazı

Tek mutex'in problemi

16 thread aynı anda `al/koy` çağırsa bile sıraya girer. Tek mutex paralelliği öldürür.

Sharding (Bölmeleme)

Sözlüğü N parçaya böl, her parçaya ayrı mutex ver. `hash(k) % N` ile bölme seç.

```
struct Bolme {
    mutable std::mutex m;
    std::unordered_map<Key, Value> veri;
};
std::deque<Bolme> bolmeler_;      // vector DEGIL!
```

Neden `std::deque`, `std::vector` Değil?

`std::mutex` taşınamaz

- `std::vector<Bolme>` büyürken iç dizisini taşır
- Mutex'in `= delete` olan kopya/taşıma constructor'ı yüzünden derleme hatası
- `std::deque` blok-blok bellek tutar; mevcut elemanları taşımaz

Alternatif

`std::vector<std::unique_ptr<Bolme»` – vector pointer'ları taşır;
Bolme yer değiştirmez.

shared_mutex: Okuyucu/Yazıcı Kilidi

Gözlem

OBYS'de okuma:yazma oranı tipik **100:1**. Çoklu okuma paralel yapılabilir; yazma tek başına olmalı.

```
mutable std::shared_mutex sm_;

T oku() const {
    std::shared_lock g(sm_);          // shared
    return /* ... */;
}

void yaz(T x) {
    std::unique_lock g(sm_);          // exclusive
    /* ... */
}
```

Dikkat

Kısa kritik bölgelerde `shared_mutex`, `mutex`'ten yavaş olabilir. Bench yapmadan tercih etme.

Yazıcı Açlığı ve Kilit Yükseltmesi

Yazıcı Açlığı

Sürekli okuyucu geliyorsa yazıcı asla kilit alamayabilir. libstdc++ ve Windows yazıcı önceliği uygular; pratikte sorun değil.

Kilit Yükseltmesi YOK

Shared kilitten unique kilide geçiş C++'ta desteklenmez. **Çift kontrol kalıbı** kullan:

1. Shared al, ihtiyaç varsa serbest bırak
2. Unique al, durumu tekrar kontrol et
3. Hâlâ gerekiyorsa yaz

Ünite 2: Eşzamanlı Dosya İşlemleri

Ünite 2 – Öğrenme Hedefleri

Bu ünitede öğreneceğin

- I/O bound vs CPU bound iş ayrımı
- Üretici-tüketici boru hattı (pipeline)
- Dosya chunking ve satır hizalaması
- Paralel yazma: tek yazıcı vs parça birleştirme
- Toplu içe/dışa aktarma deseni (batch)
- C++20 `jthread` + `stop_token` ile iptal

I/O Bound vs CPU Bound

I/O Bound

Programın çoğu zamanı disk/ağ beklemesinde. Paralelleştirme yalnızca I/O cihazı destekliyorsa kazanç verir.

CPU Bound

Programın çoğu zamanı hesaplamada. Çekirdek sayısı kadar hızlanma mümkün.

CSV İçe Aktarma

Okuma I/O bound, parsing CPU bound. Boru hattı (1 okuyucu + N parser) ideal!

Boru Hattı: 1 Üretici + N Tüketici

```
ThreadSafeQueue<std::string> kuyruk;

std::thread uretici([&] {
    std::ifstream f(yol);
    std::string s;
    while (std::getline(f, s))
        kuyruk.push(std::move(s));
    kuyruk.kapat();
});

std::vector<std::thread> isciler;
for (int i = 0; i < N; ++i) {
    isciler.emplace_back([&] {
        std::string s;
        while (kuyruk.waitAndPop(s)) {
            depo.ekle(parse(s));
        }
    });
}
```

Paralel Yazma: Tek Yazıcı Yaklaşımı

- Birden fazla işçi paralel **serileştirme** yapar
- Sonuçlar ThreadSafeQueue'ya gider
- Tek yazıcı thread sırayla dosyaya yazar

Neden tek yazıcı?

- `std::ofstream` thread-safe değil
- İç tampon paylaşılır
- Veri yarışı + iç içe geçmiş satırlar

Sıra korunumu lazımsa

Indeksli kuyruk (SiraliCikti yapısı) veya parça-blok yaklaşımı.

Batch Import: jthread + stop_token

```
struct DosyaSonuc {
    std::string yol;
    std::size_t basariliKayit = 0;
    std::vector<std::string> hatalar;
    bool tamamlandi = false;
};

BatchSonuc batchIceAktar(
    const std::vector<std::string>& yollar,
    Depo& depo, std::stop_source ss)
{
    std::vector<std::jthread> isciler;
    for (auto& y : yollar) {
        isciler.emplace_back(
            [&](std::stop_token st) {
                birDosyaAktar(y, depo, st);
            }, ss.get_token());
    }
    // jthread destructor'da otomatik join
    return /* sonuc */;
}
```

Hata İzolasyonu: Bir Dosya Diğerini Etkilemez

İlke

Her worker kendi try/catch kapsamında çalışır. İstisna `DosyaSonuc` içine yazılır, thread çökmez.

RAII ile Geçici Dosya

- `GeciciDosyaSahibi` sınıfı destructor'da silmeyi yapar
- İptal/istisnada dosya otomatik temizlenir
- Başarılıysa `tamamla()` çağrılarak silmeyi engelle

Ünite 3: İlerleme Takibi ve İptal

Ünite 3 – Öğrenme Hedefleri

Bu ünitede öğreneceğin

- Atomic sayaçla ilerleme bildirimi
- Memory order seçimi (relaxed, release, acquire)
- C++20 stop_token, stop_source, stop_callback
- jthread otomasyonu
- QProgressDialog + Worker pattern entegrasyonu
- İptal sonrası temizlik politikaları

Atomic Sayaç ile İlerleme

```
struct IlerlemeDurumu {
    std::atomic<std::size_t> islenen{0};
    std::size_t toplam = 0;
    std::atomic<bool> bitti{false};
};

void agirIs(IlerlemeDurumu& d, std::vector<X>& girdi) {
    d.toplam = girdi.size();
    for (auto& x : girdi) {
        isle(x);
        d.islenen.fetch_add(1, std::memory_order_relaxed);
    }
    d.bitti.store(true, std::memory_order_release);
}
```

- Sayaç `relaxed` – görünme sırası önemsiz
- Bitti bayrağı `release/acquire` – sayacın görünürlüğünü garanti

İlerleme Bildirim Frekansı

Çok sık emit (her satırda)

- Olay kuyruğu dolar
- UI yavaşlar
- Algı kazancı: yok

Doğru frekans (50-100 satırda 1)

- Saniyede 10-15 güncelleme yeterli
- İnsan algısı için akıcı
- CPU/event maliyeti düşük

C++20: stop_source ve stop_token

```
void uzunIs(std::stop_token st) {
    for (int i = 0; i < 1'000'000; ++i) {
        if (st.stop_requested()) {
            std::cout << "Iptal\n";
            return;
        }
        // is yap
    }
}

int main() {
    std::stop_source ss;
    std::thread t(uzunIs, ss.get_token());
    std::this_thread::sleep_for(100ms);
    ss.request_stop();
    t.join();
}
```

İşbirlikçi iptal

Tek güvenli yöntem; thread düzenli olarak kontrol eder, temiz çıkar.

jthread: Otomatik Her Şey

```
{
    std::jthread t([](std::stop_token st) {
        while (!st.stop_requested()) {
            // is yap
        }
    });
    // destructor: request_stop + join
}
```

C++20'den itibaren default tercih

- Otomatik join (terminate yok)
- İptal desteği built-in
- İstisna güvenli

stop_callback: cv Uyandırma

```
void bekle(std::stop_token st) {
    std::condition_variable cv;
    std::mutex m;

    std::stop_callback cb(st, [&] {
        cv.notify_all();
    });

    std::unique_lock kilit(m);
    cv.wait(kilit, [&] {
        return durumHazir() || st.stop_requested();
    });
}
```

Faydası

İptal istendiği anda uyuyan thread uyandırılır. Aksi halde sonsuza kadar bekleyebilir.

QProgressDialog Kurulumu

```
auto* dialog = new QProgressDialog(
    "Veriler iceri aktariliyor...",
    "Iptal", 0, toplamSatir, this);

dialog->setWindowModality(Qt::WindowModal);
dialog->setMinimumDuration(500);   // Kisa islerde acilmaz
dialog->setAutoClose(false);
```

minimumDuration = 500 ms

500 ms'den kısa süren işlerde dialog hiç açılmaz. Kullanıcı flicker (titreme) görmek yerine işin sessizce bittiğini fark eder.

Worker -> Dialog Sinyal Köprüleri

```
connect(worker, &Worker::ilerleme, dialog,
    [dialog](qsizetype i, qsizetype n) {
        dialog->setMaximum(n);
        dialog->setValue(i);
    });

connect(dialog, &QProgressDialog::canceled, worker,
        &Worker::durdur, Qt::DirectConnection);
```

- Worker → Dialog: Queued (UI thread'inde tetiklenir)
- Dialog → Worker: Direct (anında `request_stop`)

Ünite 4: OBYS Mini Proje

Ünite 4 – Mini Proje

Hedef: OBYS'nin Responsive Sürümü

Tüm I/O işlemleri arka planda; UI hiç durmaz; çoklu eş zamanlı işlemler.

Bütünleştirilen Konular

- Bölüm 1-3: STL tabanlı `Depo<T>`
- Bölüm 4-8: Qt GUI, sinyal/slot
- Bölüm 9: thread, mutex, atomic
- Bölüm 10: condition_variable, async, QThread, QtConcurrent
- Bölüm 11: thread-safe yapılar, paralel I/O, ilerleme/iptal

Üç Katmanlı Mimari

UI Katman

- AnaPencere
- Tablo, Form
- Dialog'lar
- Asla bloklanmaz

Servis Katman

- IceAktarmaServisi
- DisaAktarmaServisi
- HesaplamaServisi
- Worker thread'leri

Veri Katman

- Depo<Ogrenci>
- shared_mutex
- Çoklu okuma
- Tek yazma

Eş Zamanlı Olabilenler

Aynı anda olabilir

- İki ayrı CSV içe aktarma (depo paylaşılır)
- İçe aktarma + UI listeleme
- İçe aktarma + manuel öğrenci ekleme
- GPA hesaplama + dışa aktarma (ikisi okuma)

UI mantığında engellenir

- İki paralel GPA hesaplama (anlamsız tekrar)
- Aynı dosyaya iki dışa aktarma (overwrite çakışması)
- Veri sıfırlama + içe aktarma (mantık çelişkisi)

Thread-Safe Depo Implementasyonu

```
template <typename T>
class Depo {
    mutable std::shared_mutex sm_;
    std::unordered_map<int, std::shared_ptr<T>> kayitlar_;
public:
    std::shared_ptr<T> bul(int id) const {
        std::shared_lock g(sm_);          // shared
        auto it = kayitlar_.find(id);
        return (it != kayitlar_.end()) ? it->second : nullptr;
    }
    void ekle(int id, std::shared_ptr<T> kayit) {
        std::unique_lock g(sm_);          // exclusive
        kayitlar_[id] = std::move(kayit);
    }
    // ... tumKayitlar, sil, boyut, guncelle ...
};
```

Önemli

Arayüz Bölüm 1'deki ile aynı. Üst kod hiç değişmedi!

Servis Sınıfı - Worker Pattern

```
class IceAktarmaServisi : public QObject {
    Q_OBJECT
    std::shared_ptr<Depo<Ogrenci>> depo_;
    std::stop_source stop_;
public slots:
    void csvIceAktar(const QString& yol);
    void durdur() { stop_.request_stop(); }
signals:
    void ilerleme(qsizetype, qsizetype);
    void durum(const QString&);
    void hata(const QString&);
    void tamamlandi(qsizetype basarili, qsizetype hata);
};
```

- QObject türevi – moveToThread ile worker thread'e taşınır
- İptal için stop_source taşır
- Sinyaller UI'ya geri akar

Ana Pencere Akışı

```
void AnaPencere::csvIceAktarTetikle() {
    QString yol = QFileDialog::getOpenFileName(...);

    auto* thread = new QThread(this);
    auto* worker = new IceAktarmaServisi(depo_);
    worker->moveToThread(thread);

    auto* dialog = new QProgressDialog(...);
    // ... sinyal/slot baglantilari ...

    connect(thread, &QThread::started, worker, [worker, yol] {
        worker->csvIceAktar(yol);
    });

    thread->start();
}
```

Test Senaryoları

50.000 satırlık CSV içe aktarma

- 4-5 saniyede biter (eskiden 20-30 sn)
- UI hiç donmaz
- İlerleme çubuğu her saniye birkaç kez güncellenir

Eş zamanlı manuel giriş

- İçe aktarma sürerken yeni öğrenci formu açılır
- 5 öğrenci giriş yapılabilir
- Çakışma yok; hepsi başarıyla eklenir

İptal testi

- 100K satır iptal'e basıldı
- 1-2 saniyede dialog kapanır

Performans Karşılaştırması

Bench (8 çekirdekli makine)

İşlem	Eski	Yeni
50K CSV import	18 s	4.2 s
10K dışa aktarma	1.2 s	0.3 s
10K GPA hesaplama	32 s	4.8 s
UI tepki süresi	dondu	< 50 ms

Sonuç

4-7 kat hızlanma + UI canlılığı. Kullanıcı deneyimi nitel olarak farklı kategori.

Bu Hafta Topladıklarımız

Tasarım kalıpları

- ThreadSafeQueue, ConcurrentMap (sharded), shared_mutex
- Üretici-tüketici boru hattı
- Batch import/export deseni
- Worker pattern + QProgressDialog

Modern C++20 araçları

- std::shared_mutex + shared_lock
- std::stop_source / std::stop_token / std::stop_callback
- std::jthread (otomatik join + iptal)
- std::atomic + memory_order

Yaygın Tuzaklar – Bölüm 11 Özeti

Karşına çıkacak yanlışlar

- Arayüz yarışı (kombineli olmayan metodlar)
- Vector içinde mutex
- shared_mutex'i her yere koymak
- İptal kontrolünü atlamak
- İlerleme sinyalini her satırda emit
- RAII'siz geçici dosya
- Worker'ı UI thread'inde tutmak

Doğrulama Aracı

ThreadSanitizer (TSan): `g++ -std=c++20 -fsanitize=thread`

Bir Sonraki Hafta: Ağ Programlama

Bölüm 12: Ağ Temelleri

- TCP/IP, OSI modeli
- Soket programlama (`socket`, `bind`, `listen`)
- Qt Network: `QTcpSocket`, `QTcpServer`
- Asenkron ağ I/O

Eşzamanlılık becerilerimiz devam edecek

Çok istemcili sunucu = thread-safe veri katmanı + worker thread'ler. Bu hafta öğrendiklerimiz Bölüm 13'te OBYS sunucusuna dönüşecek.

Sorularınız?

Bölüm 11 – Eşzamanlılık Desenleri ve OBYS
Yazılım Geliştirme II